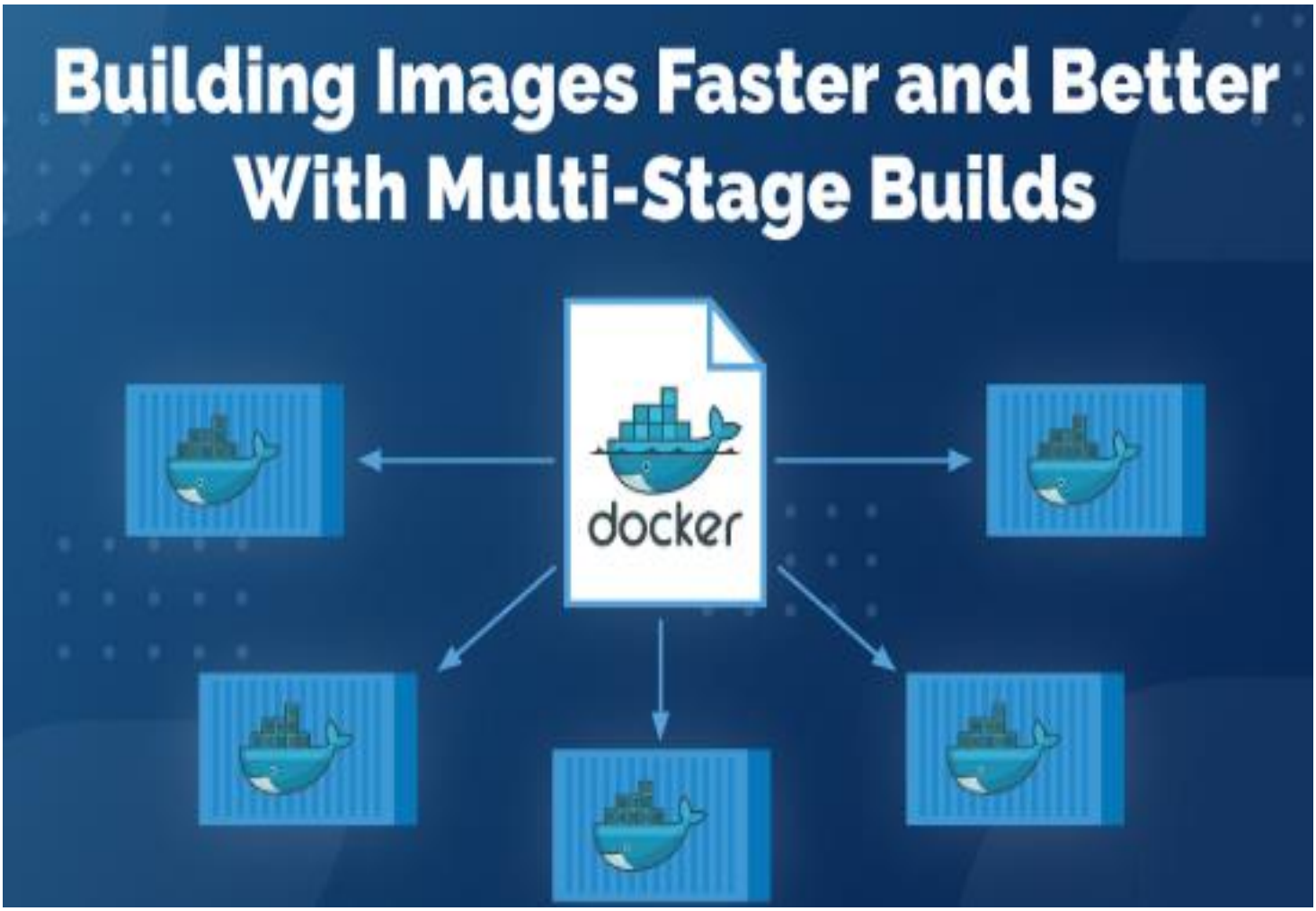
Multi-stage Docker Build

Multi-stage Docker Build is a technique in Docker that enables you to create smaller and more efficient container images by dividing the build process into multiple stages. Each stage is defined within the same Dockerfile, but only the final stage contributes to the resulting image. This approach is particularly useful for eliminating unnecessary build dependencies and intermediate files from the final image.



Key Benefits:

1. **Smaller Images:** Keeps the final image lightweight by excluding unnecessary dependencies and tools used during the build process.
2. **Security:** Reduces the attack surface by not including sensitive or unnecessary files in the final image.
3. **Simplicity:** Combines multiple build steps in a single Dockerfile, improving maintainability.

Explanation with Example:

Let's assume you have a simple application written in **Go**.

Application Structure:

- `main.go`: The source code.
- `Dockerfile`: The Docker build configuration.

Multi-Stage Dockerfile:

```dockerfile
# Stage 1: Build Stage
FROM golang:1.20 AS builder
WORKDIR /app
COPY main.go .
RUN go build -o main .

# Stage 2: Final Image
FROM alpine:3.18
WORKDIR /app
COPY --from=builder /app/main .
CMD ["./main"]
```

Step-by-Step Breakdown:

1. **Stage 1: Build Stage (`builder`)**
   - A **base image** with all build tools (e.g., `golang:1.20`) is used.
   - Source code (`main.go`) is copied into the container.
   - The application is compiled (`go build`), creating the `main` binary.
2. **Stage 2: Final Image**
   - A **minimal base image** (`alpine`) is used for the final container.
   - The compiled binary (`main`) is copied from the `builder` stage.
   - The final image contains only the application binary and necessary files, excluding the Go compiler and build tools.

Build and Run the Docker Image:

1. Build the image

```bash
docker build -t myapp .
```

2. Run the container

```bash
docker run --rm myapp
```

Why Use Multi-Stage Builds?

- **Efficiency:** Intermediate layers are discarded, keeping the final image lean.
- **Clean Separation:** Separate build and runtime environments.
- **Versatility:** Combine multiple stages for complex workflows (e.g., building assets in one stage and the app in another).